

IMPORTANT

CSS Practices To Avoid As Developer



Must Check



Set Margins Or Padding And Then Reset Them



```
.item {  
  margin-right: 1.6rem;  
}  
  
.item:last-child {  
  margin-right: 0;  
}
```



```
.item:not(:last-child) {  
  margin-right: 1.6rem;  
}
```

I Often See People Set Margins Or Padding For All Elements
And Then Reset Them For The First Or Last Element.

Set Display: Block For Flex Items



```
.parent {  
  display: flex;  
}  
  
.child {  
  display: block;  
}
```



```
.parent {  
  display: flex;  
}
```

When Using Flexbox, It Is Important To Remember That When You Create A Flex Container (Add Display: Flex), All Children (Flex Items) Become Blockified.

Use Width: 100% For Block Elements



```
index.html →
<div class="parent">
  <div class="child">Item 1</div>
  <div class="child">Item 2</div>
  <div class="child">Item 3</div>
  <div class="child">Item 4</div>
</div>

style.css →
<style>
  .parent {
    display: flex;
    flex-wrap: wrap;
  }

  .child {
    width: 100%;
  }

  @media (min-width: 1024px) {
    .child {
      width: 25%;
    }
  }
</style>
```



```
index.html →
<div class="parent">
  <div class="child">Item 1</div>
  <div class="child">Item 2</div>
  <div class="child">Item 3</div>
  <div class="child">Item 4</div>
</div>

style.css →
<style>
@media (min-width: 1024px) {
  .parent {
    display: flex;
    flex-wrap: wrap;
  }

  .child {
    width: 25%;
  }
}
</style>
```

Use Transform: Translate (-50%, -50%) To Center



```
.parent {  
  position: relative;  
}  
  
.child {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%,  
-50%);  
}
```



```
.parent {  
  display: flex;  
}  
  
.child {  
  margin: auto;  
}
```

Verticle Center

In Particular, One Solution Was To Use A Combination Of Absolute Positioning And The Transform Property. This Technique Caused Blurry Text Issues In Chromium-Based Browsers.

We Can Use Margin: Auto Inside A Flex Container And The Browser Will Center The Element. Just Two Properties And That's It.

!Important Is Unnecessarily Used In Most Places:



```
.parent {  
  background: red !important;  
  color: #fff !important;  
  font-weight: bold !important;  
}
```



```
.parent {  
  background: red;  
  color: #fff;  
  font-weight: bold;  
}
```

In CSS Styles Are Cascading Or Overlapping From The Top Of One To Another. But, When You Bring Down !Important In Between, It Will Dominate The Others And As The Name Suggests It Will Establish That It Is Important Over The Others, Just Dont Use Unnecessarily.

Inline Styles In CSS 🙄



```
<button style="outline: none;">Login</button>
```



```
<button class="login_btn">Login</button>

/*style.css*/
.login_btn{
  outline: none;
}
```


Inline Styles In CSS 🙄



```
<button style="outline: none;">Login</button>
```



```
<button class="login_btn">Login</button>

/*style.css*/
.login_btn{
  outline: none;
}
```